

R: limpieza, manipulación y exploración de datos

6to encuentro: Manipulación de datos III

Jonatán Perren

12 de mayo de 2026

Seguimos construyendo sobre la gramática de datos. En este encuentro completamos las herramientas que extienden lo que ya sabemos de `mutate()`, y arrancamos con el tratamiento especializado de fechas.

¿Qué vemos hoy?

- Operaciones condicionales vectorizadas: `if_else()`, `case_when()`, `case_match()`
- Operar sobre múltiples columnas a la vez: `across()`
- Fechas en R: el paquete `lubridate`

Todo esto vive dentro de `mutate()` y `summarise()`, así que el punto de partida es exactamente lo que ya sabemos hacer.

if_else(): la versión tidyverse de ifelse()

Ya vimos `ifelse()` de base R en el segundo encuentro. `dplyr::if_else()` hace lo mismo con dos ventajas importantes:

- **Tipado estricto:** los valores para `TRUE` y `FALSE` deben ser del mismo tipo. Previene errores silenciosos.
- **Argumento missing:** permite especificar explícitamente qué devolver cuando la condición es `NA`.

`ifelse(TRUE, 1L, "texto")` devuelve "1" sin ningún aviso. `if_else()` da error directamente para que no pase desapercibido.

```
1 if_else(condition, true, false, missing = NULL)
```

Creemos un dataset de empleados y apliquemos `if_else()`.

```
1 empleados <- tibble(  
2   nombre   = c("Ana", "Carlos", "María", "Juan", "Laura", "Pedro"),  
3   area     = c("ventas", "sistemas", "ventas", "rrhh", "sistemas", "rrhh"),  
4   provincia = c("Santa Fe", "Buenos Aires", "Córdoba",  
5                 "Mendoza", "Buenos Aires", "Santa Fe"),  
6   sueldo   = c(85000, 120000, 92000, 78000, 135000, 71000),  
7   antigüedad = c(3, 7, 2, 5, 10, 1)  
8 )  
9  
10 empleados >  
11   mutate(categoria = if_else(sueldo > 100000, "Senior", "Junior"))
```

```
> empleados > mutate(categoria = if_else(sueldo > 100000, ...))  
# A tibble: 6 x 6  
  nombre sueldo categoria  
  <chr>   <dbl>   <chr>  
1 Ana     85000   Junior  
2 Carlos 120000   Senior  
3 María   92000   Junior  
4 Juan    78000   Junior  
5 Laura  135000   Senior  
6 Pedro   71000   Junior
```

case_when(): múltiples condiciones

Cuando tenemos más de dos categorías, anidar `if_else()` se vuelve ilegible.

`case_when()` es el reemplazo elegante: evalúa condiciones en orden y devuelve el primer resultado que sea **TRUE**.

```
1 mutate(nueva_col = case_when(  
2   condicion_1 ~ resultado_1,  
3   condicion_2 ~ resultado_2,  
4   condicion_3 ~ resultado_3,  
5   .default    = resultado_resto # equivale al "else" final  
6 ))
```

Las condiciones se evalúan de arriba hacia abajo: la primera que sea **TRUE** gana. Los casos no cubiertos devuelven **NA** salvo que uses `.default`. En versiones anteriores de dplyr, el equivalente era **TRUE ~ resultado**.

```

1 empleados >
2   mutate(banda = case_when(
3     sueldo ≥ 120000 ~ "A",
4     sueldo ≥ 90000  ~ "B",
5     sueldo ≥ 70000  ~ "C",
6     .default       = "D"
7   ))

```

```

> empleados > mutate(banda = case_when( ... ))
# A tibble: 6 x 6
  nombre sueldo  banda
  <chr>   <dbl> <chr>
1 Ana     85000  C
2 Carlos 120000  A
3 María  92000  B
4 Juan   78000  C
5 Laura 135000  A
6 Pedro  71000  C

```

También podemos combinar condiciones sobre múltiples columnas en una misma expresión:

```

1 empleados >
2   mutate(bono = case_when(
3     area == "sistemas" & antigüedad ≥ 5 ~ sueldo * 0.20,
4     antigüedad ≥ 5                      ~ sueldo * 0.10,
5     .default                      = 0
6   ))

```

case_match(): recodificar por valor exacto

Cuando la condición es solo `variable == "valor"`, `case_match()` es más conciso. En lugar de escribir la comparación completa, se lista directamente el valor a matchear.

```
1 # Con case_when (repetitivo)
2 mutate(region = case_when(
3   provincia == "Buenos Aires" ~ "Centro",
4   provincia == "Santa Fe"     ~ "Centro",
5   provincia == "Córdoba"      ~ "Centro",
6   provincia == "Mendoza"      ~ "Cuyo",
7   .default                    = "Otra"
8 ))
9
10 # Con case_match (equivalente y más compacto)
11 mutate(region = case_match(
12   provincia,
13   c("Buenos Aires", "Santa Fe", "Córdoba") ~ "Centro",
14   "Mendoza"                                ~ "Cuyo",
15   .default = "Otra"
16 ))
```

```

1 empleados ▶
2   mutate(region = case_match(
3     provincia,
4     c("Buenos Aires", "Santa Fe", "Córdoba") ~ "Centro",
5     "Mendoza" ~ "Cuyo",
6     .default = "Otra"
7   )) ▶
8   select(nombre, provincia, region)

```

```

> empleados ▶ mutate(region = case_match(provincia, ...))
# A tibble: 6 x 3
  nombre provincia region
  <chr>   <chr>   <chr>
1 Ana     Santa Fe   Centro
2 Carlos  Buenos Aires Centro
3 María   Córdoba   Centro
4 Juan    Mendoza   Cuyo
5 Laura   Buenos Aires Centro
6 Pedro   Santa Fe   Centro

```

`case_match()` es ideal para tablas de recodificación: países a regiones, códigos a etiquetas, abreviaturas a nombres completos.

across(): la misma operación en múltiples columnas

Situación muy común: aplicar la misma transformación a varias columnas. Sin `across()` habría que repetir el código columna por columna.

```
1 mediciones <- tibble(  
2   id = c(1, 2, 3),  
3   x  = c(1.234, 2.567, 3.891),  
4   y  = c(4.112, 5.436, 6.789),  
5   z  = c(7.001, 8.345, 9.678)  
6 )  
7  
8 # Sin across: repetitivo y propenso a errores de tipeo  
9 mediciones > mutate(x = round(x, 2), y = round(y, 2), z = round(z, 2))  
10  
11 # Con across: una sola línea  
12 mediciones > mutate(across(c(x, y, z), ~round(.x, 2)))
```

`across()` recibe dos argumentos:

- **Columnas:** un vector de nombres, o selectores como `where(is.numeric)`, `starts_with()`, etc.
- **Función:** puede ser una función nombrada (`mean`, `round`) o una función anónima (incluso con la sintaxis `~round(.x, 2)`). Si no sabés qué es una función anónima o las formas de escribirlas, revisá el **machete de funciones y funciones anónimas**.

across() dentro de mutate()

```
1 # Redondear todas las columnas numéricas a 1 decimal
2 iris > mutate(across(where(is.numeric), ~round(.x, 1)))
3
4 # Generar columnas nuevas sin pisar las originales, usando .names
5 empleados >
6   mutate(across(sueldo, ~.x / 1050, .names = "{.col}_usd"))
```

across() dentro de summarise()

```
1 # Promedio de todas las columnas numéricas, por área
2 empleados >
3   group_by(area) >
4   summarise(across(where(is.numeric), mean))
```

```
> empleados > group_by(area) > summarise(across(where(is.numeric), mean))
# A tibble: 3 x 3
  area      sueldo antigüedad
<chr>    <dbl>    <dbl>
1 rrhh      74500         3
2 sistemas 127500        8.5
3 ventas   88500        2.5
```

Múltiples funciones con across()

Si querés aplicar más de una función, pasás una lista nombrada. Los nombres de la lista se usan para construir los nombres de las columnas resultantes.

```
1 empleados ►  
2 summarise(across(  
3   where(is.numeric),  
4   list(promedio = mean, maximo = max, minimo = min)  
5 ))
```

```
> empleados ► summarise(across(where(is.numeric), list(...)))  
# A tibble: 1 x 6  
  sueldo_promedio sueldo_maximo sueldo_minimo antiguedad_promedio ...  
    <dbl>          <dbl>          <dbl>          <dbl>  
1      96833        135000         71000          4.67
```

El nombre de cada columna nueva combina el nombre original con el nombre de la función: `suelo_promedio`, `suelo_maximo`, etc. Esto se puede personalizar con el argumento `.names`.

Las fechas son uno de los tipos de datos más engañosos. Si cargás una base del INDEC, del Banco Mundial o de AFIP, las fechas pueden llegar como texto ("**01/01/2023**"), como número (**44927**, el formato interno de Excel), o en distintos formatos según el idioma del sistema.

Una fecha guardada como texto es solo texto: no podés calcular diferencias, extraer el mes, ni ordenar cronológicamente de forma confiable.

R tiene dos tipos nativos para representar fechas:

- **Date**: solo fecha (año, mes, día). Suficiente para la mayoría del análisis económico.
- **POSIXct**: fecha y hora. Para datos de alta frecuencia (precios intradiarios, registros horarios).

Internamente, un **Date** es simplemente un entero: la cantidad de días desde el 1 de enero de 1970. Esto lo hace apto para operaciones aritméticas directas.

El paquete lubridate: parsear fechas

lubridate simplifica el trabajo con fechas. Sus funciones de parseo se nombran según el orden de los componentes (**year**, **month**, **day**) y toleran distintos separadores. Podés encontrar un machete de lubridate [acá](#) o el cheatsheet oficial en la figura.



```
1 library(lubridate)
2
3 ymd("2023-07-15")      # año-mes-día: formato ISO (APIs, bases internacionales)
4 dmy("15/07/2023")      # día/mes/año: formato argentino
5 mdy("July 15, 2023")   # mes día, año: formato anglosajón
6 ymd(20230715)          # sin separadores: también funciona
```

```
> dmy("15/07/2023")
[1] "2023-07-15"
> class(dmy("15/07/2023"))
[1] "Date"
```

Elegí la función según cómo llegan tus datos, no según cómo querés que se muestren. El formato de salida siempre es YYYY-MM-DD (ISO 8601) independientemente del input.

Creemos un dataset de precios minoristas con fechas reales:

```
1 ipc ← tibble(  
2   fecha      = dmy(c("01/01/2023", "01/02/2023", "01/03/2023",  
3                     "01/04/2023", "01/05/2023", "01/06/2023")),  
4   indice     = c(248.3, 276.5, 300.7,  
5                 330.4, 358.6, 394.2),  
6   categoria  = c("alimentos", "alimentos", "indumentaria",  
7                 "alimentos", "indumentaria", "alimentos")  
8 )
```

```
> ipc  
# A tibble: 6 x 3  
  fecha      indice categoria  
  <date>      <dbl> <chr>  
1 2023-01-01  248. alimentos  
2 2023-02-01  277. alimentos  
3 2023-03-01  301. indumentaria  
4 2023-04-01  330. alimentos  
5 2023-05-01  359. indumentaria  
6 2023-06-01  394. alimentos
```

Extraer componentes de una fecha

Una vez que la columna es de tipo **Date**, podemos extraer cualquier componente con funciones específicas. Son especialmente útiles dentro de **mutate()** para crear variables de agrupamiento temporal.

Función	Devuelve	Notas
year()	Año	2023
month()	Mes (número)	1 a 12
month(label = TRUE)	Mes (abreviado)	"ene.", "feb.", ...
day()	Día del mes	1 a 31
wday()	Día de la semana	1 = domingo
quarter()	Trimestre	1 a 4
yday()	Día del año	1 a 365

```
1 ipc >
2   mutate(
3     anio      = year(fecha),
4     mes       = month(fecha),
5     trimestre = quarter(fecha)
6   )
```

La función **quarter()** es sumamente versátil. Deberías investigarla .

Como las fechas son números enteros internamente, las operaciones aritméticas funcionan de forma natural.

Diferencia entre fechas

```
1 fecha_ingreso <- dmy("15/03/2018")
2 fecha_hoy     <- today()
3
4 fecha_hoy - fecha_ingreso      # resultado en días (class "difftime")
5 as.numeric(fecha_hoy - fecha_ingreso) / 365 # en años aproximados
```

```
> fecha_hoy - fecha_ingreso
Time difference of 2933 days
```

Variación periódica con lag()

Uno de los usos más frecuentes en economía: calcular la variación porcentual respecto al período anterior.

```
1 ipc >
2   arrange(fecha) >
3   mutate(var_mensual = (indice / lag(indice) - 1) * 100)
```



```
> ipc > arrange(fecha) > mutate(var_mensual = (indice / lag(indice) - 1) * 100)
# A tibble: 6 x 4
  fecha      indice var_mensual
  <date>      <dbl>      <dbl>
1 2023-01-01    248.         NA
2 2023-02-01    277.        11.4
3 2023-03-01    301.         8.7
4 2023-04-01    330.         9.9
5 2023-05-01    359.         8.5
6 2023-06-01    394.         9.9
```

Redondear y truncar fechas

`floor_date()` y `ceiling_date()` truncan o redondean la fecha a la unidad de tiempo indicada. Muy útil para agrupar datos diarios en meses o trimestres.

```
1 d <- dmy("17/07/2023")
2
3 floor_date(d, unit = "month")    # -> 2023-07-01 (inicio del mes)
4 floor_date(d, unit = "quarter") # -> 2023-07-01 (inicio del trimestre)
5 ceiling_date(d, unit = "month") # -> 2023-08-01 (inicio del mes siguiente)
```

Juntemos todo. Queremos un resumen trimestral de la inflación por categoría, clasificando cada mes según la intensidad de la variación.

```
1 ipc >
2   arrange(categoria, fecha) >
3   group_by(categoria) >
4   mutate(
5     var_mensual = (indice / lag(indice) - 1) * 100,
6     trimestre   = quarter(fecha),
7     nivel       = case_when(
8       var_mensual ≥ 10 ~ "Alta",
9       var_mensual ≥ 5  ~ "Moderada",
10      .default       = "Baja"
11    )
12  ) >
13  ungroup() >
14  group_by(categoria, trimestre) >
15  summarise(
16    across(var_mensual, list(promedio = mean, max = max),
17           na.rm = TRUE),
18    .groups = "drop"
19  )
```

```
> ipc > arrange(categoria, fecha) > group_by(categoria) > mutate(...) > ...
# A tibble: 4 x 4
  categoria trimestre var_mensual_promedio var_mensual_max
  <chr>         <int>         <dbl>         <dbl>
1 alimentos             1             11.4             11.4
2 alimentos             2             19.4             19.5
3 indumentaria          1              NaN             -Inf
4 indumentaria          2             19.3             19.3
```

La fila 3 tiene NaN y -Inf porque indumentaria solo tiene una observación en el Q1, y esa observación tiene NA en var_mensual (es el primer valor del grupo, no tiene lag). Al aplicar na.rm = TRUE sobre un vector de puros NA, mean() devuelve NaN y max() devuelve -Inf.

1. En `starwars`, agregá una columna `grande` que valga `"sí"` si el personaje mide más de 180 cm, `"no"` si mide menos, y `"desconocido"` si la altura es `NA`.
2. En `mtcars`, agregá una columna `tipo_motor` que distinga los autos con 8 cilindros del resto.
3. En `starwars`, clasificá la altura en tres categorías: `"bajo"` (menos de 160 cm), `"medio"` (160–190 cm) y `"alto"` (más de 190 cm). Los `NA` deben quedar como `NA`.
4. En `mtcars`, clasificá el consumo en: `"económico"` (mpg > 25), `"normal"` (mpg entre 15 y 25) y `"consumidor"` (mpg < 15).
5. En `starwars`, creá una columna `zona_galactica` asignando al menos 5 planetas a distintas zonas. Los planetas no asignados deben quedar como `"Desconocida"`.
6. Creá un tibble con 8 provincias argentinas y agregá una columna `region` que las clasifique en NOA, NEA, Cuyo, Centro o Patagonia.
7. En `starwars`, recodificá `species` para quedarte solo con `"Human"`, `"Droid"` y `"Otro"` (todos los demás).

8. En `iris`, redondeá todas las columnas numéricas a 1 decimal.
9. En `starwars`, eliminá las filas con `NA` en `height` o `mass`, y luego estandarizá ambas columnas (restando la media y dividiendo por el desvío estándar).
10. Calculá el promedio y la mediana de todas las variables numéricas de `mtcars` en una sola operación.
11. En `iris`, calculá el promedio de cada variable numérica para cada especie.
12. En `iris`, creá versiones escaladas de todas las variables numéricas. Los nombres de las nuevas columnas deben terminar en `_scaled`.
13. Convertí los siguientes textos a fechas y verificá que R las reconozca correctamente: `"2023-03-25"`, `"25/03/2023"`, `"March 25 2023"`.
14. Convertí el vector `c("01012022", "15062023", "31122024")` a fechas. ¿Cuál es el orden de los componentes?

15. Retomá el dataset `ipc` de la clase y extraé el año, el mes y el trimestre de cada observación en columnas separadas.
16. ¿Cuántos días pasaron entre el 25 de mayo de 1810 y el 9 de julio de 1816? ¿Y cuántos años aproximados?
17. Creá un tibble con 12 filas que represente el IPC mensual de 2023 (valores inventados) y calculá la variación respecto al mes anterior.
18. Sobre el dataset anterior, calculá también la variación respecto al mismo mes del año anterior.
19. ¿Cuántos días faltan para el 25 de mayo de 2026?
20. Agregá al dataset `ipc` una columna con el primer día del trimestre al que pertenece cada observación.
21. En `starwars`, clasificá el año de nacimiento (`birth_year`) en décadas:
"0-50 DBY", "50-100 DBY", "100+ DBY". Los NA deben quedar como NA.

22. En `mtcars`, clasificá los autos según la cantidad de cilindros en "`chico`", "`mediano`" o "`grande`", y luego calculá el promedio de todas las variables numéricas para cada categoría.
23. Creá un dataset de exportaciones con columnas `fecha` (texto en formato "`dd/mm/aaaa`"), `producto`, `toneladas` y `precio_usd`. A partir de él:
- 1) Convertí la fecha al tipo correcto.
 - 2) Extraé el mes y el trimestre.
 - 3) Calculá el ingreso total por fila.
 - 4) Clasificá el ingreso en categorías.
 - 5) Resumí el ingreso por trimestre.
24. Retomá el dataset de variación mensual del ejercicio 17. Clasificá cada mes según si la inflación se aceleró o desaceleró respecto al mes anterior. Pista: necesitás comparar la variación actual con la del período previo.

25. Del dataset `starwars`, quedáte solo con los personajes humanos que tengan altura y masa conocidas. Calculá el IMC, clasificálo en categorías («bajo peso», «normal», «sobrepeso», «obesidad»), y obtené el IMC promedio y la cantidad de personajes por categoría, ordenado de mayor a menor IMC.
26. En `mtcars`, convertí el consumo a km/l, clasificá cada auto según su eficiencia, y encontrá los 3 autos más eficientes dentro de cada grupo de cilindros.
27. Creá un dataset de 10 empleados con columnas `nombre`, `area`, `fecha_ingreso` (texto en formato argentino) y `suel`do. Convertí la fecha, calculá la antigüedad en años, clasificá a cada empleado según su antigüedad, y obtené el sueldo promedio por área y categoría de antigüedad.
28. En `iris`, filtrá las flores con pétalos más largos que el promedio general, y calculá el promedio de todas las variables numéricas para cada especie dentro de ese subconjunto.
29. En `starwars`, filtrá los personajes con año de nacimiento conocido. Clasificálos por década de nacimiento y, para cada década, calculá el promedio de altura y masa. Mostrá solo las décadas con al menos 3 personajes.

30. En **mtcars**, clasificá los autos según cilindros. Luego, para cada categoría, calculá el promedio, máximo y mínimo de todas las variables numéricas en una sola operación. Ordená el resultado por consumo promedio descendente.
31. Creá un dataset de precios mensuales de tres productos durante 2023 (36 filas en total: 12 meses $\times$ 3 productos). Convertí las fechas, calculá la variación mensual de cada producto, clasificá cada mes como «suba», «baja» o «estable», y resumí cuántos meses cayó en cada categoría por producto.
32. En **starwars**, seleccioná nombre, especie, altura, masa y mundo natal. Filtrá los personajes con altura entre 150 y 200 cm y masa conocida. Clasificá la especie en «Human», «Droid» u «Otro». Contá cuántos personajes hay por especie y clasificación de altura, y quedáte solo con las combinaciones que tengan más de un personaje.
33. Creá un dataset de ventas diarias durante el primer trimestre de 2024 (al menos 30 filas). Convertí las fechas, extraé el mes y la semana del año, calculá la variación diaria respecto al día anterior, y resumí el ingreso total y la variación promedio por mes.
34. En **mtcars**, clasificá cada auto en «potente» (más de 200 hp) o «estándar». Para cada grupo, calculá el promedio, el máximo y el mínimo de todas las variables numéricas en una sola operación, y ordená los resultados por consumo promedio descendente.

Continuaremos trabajando con `flights` de `nycflights13`. El objetivo es integrar las tres herramientas del encuentro: clasificación condicional (`if_else()`, `case_when()`, `case_match()`), operaciones sobre múltiples columnas con `across()`, y manejo de fechas con `lubridate`.

1. En un único `mutate()`, agregá dos columnas nuevas:
 - `salio_tarde`: `TRUE` si `dep_delay > 0`, `FALSE` si es menor o igual a cero, y `NA` cuando la demora sea desconocida.
 - `categoria_demora`: «En hora» ($\text{demora} \leq 0$), «Leve» (1–30 min), «Moderada» (31–120 min), «Grave» (más de 120 min). Los registros con `dep_delay` igual a `NA` deben quedar como `NA`.
2. Usá `make_date(year, month, day)` de `lubridate` para construir una columna fecha de tipo `Date`. Luego, en el mismo `mutate()`, extraé el día de la semana con `wday(label = TRUE)` y el trimestre con `quarter()`. ¿En qué día de la semana hay más vuelos? ¿Ese patrón varía según el aeropuerto de origen?
3. Calculá, agrupando por `origin`, la media de las columnas de demora en una única operación con `across()` dentro de `summarise()`. ¿Qué aeropuerto tiene menor demora media de salida?

4. Usá `case_match()` para crear una columna `aerolinea` a partir de `carrier`, asignando el nombre completo a por lo menos 6 aerolíneas (podés consultar los códigos con `nycflights13::airlines`). Las restantes deben quedar como «Otra». Luego, agrupando por `aerolinea`, calculá la cantidad de vuelos y la demora media de salida.
5. Construí un único pipeline que integre todo lo anterior. Partiendo de `flights`: construí `fecha` con `make_date()`, extraé el trimestre, creá `salio_tarde` con `if_else()`, clasificá `dep_delay` con `case_when()` en las cuatro categorías del punto 1, recodificá `carrier` con `case_match()` al nombre completo, y calculá por aerolínea y trimestre: cantidad de vuelos, demora media de salida y proporción de vuelos que salieron a tiempo. Guardá el resultado en `resumen_vuelos`.
6. para que pienses vos la implementación 😊 ¿Las aerolíneas con más vuelos son también las más puntuales? ¿En qué trimestre del año se concentran las mayores demoras?